

Laboratório 7 — Grafos: Sistema de Evacuação Inteligente

1. Identificação

- **Aluno:** Daniel Santana Souza — **Matrícula:** 2310995
- **Disciplina:** INF 1010 — Estruturas de Dados Avançadas (PUC-Rio)
- **Trabalho:** Grafos — busca em profundidade (DFS), busca em largura (BFS) e componentes conexas aplicadas a um sistema de evacuação.

2. Objetivo

O trabalho consiste em modelar o mapa de um edifício como um grafo e usar algoritmos de busca para apoiar a evacuação do prédio em emergências. Cada ambiente (recepção, laboratórios, corredores, auditório, escada e a saída de emergência) vira um vértice, e cada passagem entre ambientes vira uma aresta de um grafo **não dirigido e não ponderado**. Sobre esse grafo é preciso: listar quais ambientes podem ser alcançados a partir de um ponto usando **DFS**; encontrar o caminho com o menor número de deslocamentos até a saída usando **BFS**; e, depois de um incêndio bloquear uma passagem, descobrir se ainda existe rota de fuga, quais ambientes ficaram isolados e quantas **componentes conexas o grafo passou a ter**. **O objetivo prático é implementar tudo em C com listas de adjacência**, compilar, gerar o executável e validar as respostas.

3. Estrutura do programa

O programa foi dividido em módulos, separando a estrutura de dados (o grafo e seus algoritmos) do programa que monta o mapa e roda os experimentos.

grafo.h

Cabeçalho do módulo de grafo. Define as constantes `MAX_NOME` e `MAX_VERTICES`, o tipo `Viz` (célula da lista de adjacência, no modelo visto em aula, sem peso por ser um grafo não ponderado) e o tipo `Grafo` (com o número de vértices `nv`, o número de arestas `na`, o vetor de nomes dos ambientes e o vetor `viz` de listas de adjacência). Declara também os protótipos das funções públicas.

grafo.c

Implementação do módulo. Principais funções:

- `cria_grafo()` — cria um grafo vazio.

- `indice_vertice(g, nome)` — traduz o nome de um ambiente no seu índice.
- `adiciona_vertice(g, nome)` — cadastra um ambiente (sem duplicar).
- `adiciona_aresta(g, a, b)` — cria a conexão entre dois ambientes; como o grafo é não dirigido, registra a ligação nas duas listas de adjacência.
- `remove_aresta(g, a, b)` — remove a conexão das duas listas (usada para simular o bloqueio causado pelo incêndio).
- `imprime_grafo(g)` — imprime o grafo no formato de listas de adjacência.
- `dfs(g, origem)` — **busca em profundidade** recursiva; imprime os ambientes na ordem de visita e devolve quantos foram alcançados.
- `bfs_caminho(g, origem, destino)` — **busca em largura**; usa uma fila e um vetor de predecessores para reconstruir e imprimir o caminho mínimo, devolvendo o número de deslocamentos.
- `componentes_conexas(g)` — conta e imprime as componentes conexas.
- `libera_grafo(g)` — libera toda a memória alocada.

`main.c`

Programa principal. A função `constroi_mapa()` monta o grafo do edifício com as 7 conexões do enunciado, e `main()` executa, em ordem, os quatro blocos de perguntas do laboratório, imprimindo as respostas.

4. Solução

4.1. Representação do mapa (listas de adjacência)

A partir do mapa do enunciado foram identificados 8 ambientes e 7 conexões:

```
Laboratorio1 -- Recepcao
Recepcao     -- CorredorA
CorredorA    -- Laboratorio2
Recepcao     -- Auditorio
Auditorio    -- CorredorB
CorredorB    -- Escada
CorredorB    -- Saida (Saida de Emergencia)
```

Cada conexão é guardada nas listas de adjacência dos dois ambientes que ela liga (grafo não dirigido). A representação produzida pelo programa é:

```
Grafo (8 ambientes, 7 conexoes) - listas de adjacencia:
[0] Laboratorio1      -> Recepcao
[1] Recepcao          -> Auditorio CorredorA Laboratorio1
[2] CorredorA         -> Laboratorio2 Recepcao
[3] Laboratorio2     -> CorredorA
[4] Auditorio         -> CorredorB Recepcao
[5] CorredorB         -> Saida Escada Auditorio
[6] Escada            -> CorredorB
[7] Saida             -> CorredorB
```

A ordem dos vizinhos aparece invertida em relação à inserção porque cada novo vizinho é colocado no início da lista (inserção em $O(1)$).

4.2. DFS a partir da Recepção

A DFS parte da Recepção, marca cada vértice ao visitá-lo e desce recursivamente para o primeiro vizinho ainda não visitado, retrocedendo quando não há mais para onde ir. A saída do programa é:

```
DFS a partir de "Recepcao":
```

```
Recepcao -> Auditorio -> CorredorB -> Saida -> Escada -> CorredorA -> Laboratorio2 -> Laboratorio1
```

```
Total de ambientes visitados: 8 de 8.
```

- **(a) Ambientes visitados a partir da Recepção:** todos os 8 (Recepção, Auditório, Corredor B, Saída, Escada, Corredor A, Laboratório 2 e Laboratório 1). Como o grafo original é conexo, a DFS alcança todos os ambientes a partir de qualquer ponto. (A ordem exata depende de como os vizinhos estão encadeados na lista de adjacência.)
- **(b) Por que nenhum ambiente é visitado mais de uma vez:** ao entrar em um vértice ele é imediatamente marcado como visitado, e antes de descer para um vizinho o algoritmo checa essa marca. Assim, um ambiente já marcado nunca é reentrado. Isso garante o término do algoritmo (mesmo havendo ciclos, como Recepção–Corredor A–...) e evita repetições.
- **(c) Complexidade:** usando listas de adjacência, cada vértice é visitado uma vez e cada aresta é examinada um número constante de vezes, resultando em $O(V + E)$ (V = número de vértices, E = número de arestas).

4.3. BFS — caminhos mínimos até a Saída

A BFS explora o grafo em "camadas" a partir da origem usando uma fila: primeiro os vizinhos a 1 deslocamento, depois os a 2 deslocamentos, e assim por diante. Guardando o predecessor de cada vértice, é possível reconstruir o caminho mínimo. A saída do programa é:

```
Caminho mínimo de "Laboratorio1" ate "Saida" (4 deslocamentos):
```

```
Laboratorio1 -> Recepcao -> Auditorio -> CorredorB -> Saida
```

```
Caminho mínimo de "Laboratorio2" ate "Saida" (5 deslocamentos):
```

```
Laboratorio2 -> CorredorA -> Recepcao -> Auditorio -> CorredorB -> Saida
```

- **(a/b) Caminhos mínimos e número de deslocamentos:**
 - Laboratório 1 → Saída: Laboratório 1 → Recepção → Auditório → Corredor B → Saída, 4 deslocamentos.
 - Laboratório 2 → Saída: Laboratório 2 → Corredor A → Recepção → Auditório → Corredor B → Saída, 5 deslocamentos.
- **(c) Por que a BFS é adequada para caminhos mínimos em grafos não ponderados:** ela visita os vértices em ordem crescente de distância (número de arestas) à origem. Quando um vértice é alcançado pela primeira vez, isso acontece pelo caminho com o menor número de arestas possível — exatamente a noção de caminho mínimo quando todas as arestas "custam" 1.
- **(d) A DFS encontraria o mesmo caminho?** Não necessariamente. A DFS segue

fundo por um ramo antes de retroceder e pode chegar ao destino por um caminho mais longo, dependendo da ordem dos vizinhos. Ela encontra algum caminho, mas não há garantia de que seja o mínimo; a BFS é quem garante o mínimo.

4.4. Incêndio: remoção da conexão Auditório-Corredor B

O incêndio bloqueia a passagem (Auditório, Corredor B). O programa remove essa aresta das duas listas e reanalisa o grafo:

Grafo (8 ambientes, 6 conexões) - listas de adjacência:

```
[0] Laboratorio1      -> Recepcao
[1] Recepcao         -> Auditorio CorredorA Laboratorio1
[2] CorredorA        -> Laboratorio2 Recepcao
[3] Laboratorio2     -> CorredorA
[4] Auditorio        -> Recepcao
[5] CorredorB        -> Saida Escada
[6] Escada           -> CorredorB
[7] Saida            -> CorredorB
```

- **(a) Existe caminho entre Recepção e Saída?** Não. A BFS de Recepção até Saída não encontra rota:

Não há caminho entre "Recepcao" e "Saida".

Resposta: NÃO há mais caminho.

- **(b) Ambientes isolados (da Recepção):** Corredor B, Escada e Saída de Emergência. Eles deixaram de ter ligação com o lado da Recepção.

- **(c) DFS novamente a partir da Recepção:**

DFS a partir de "Recepcao":

Recepcao -> Auditorio -> CorredorA -> Laboratorio2 -> Laboratorio1

Ambientes alcançáveis a partir da Recepcao: 5 de 8.

Agora a DFS só alcança 5 ambientes (Recepção, Auditório, Corredor A, Laboratório 2 e Laboratório 1).

- **(d) Componente conexa:** é um subconjunto máximo de vértices no qual existe um caminho entre quaisquer dois deles. "Máximo" significa que não é possível acrescentar mais nenhum vértice sem perder essa propriedade. Num grafo conexo há uma única componente; ao remover arestas o grafo pode se partir em várias.

- **(e) Quantas componentes conexas existem agora?** Duas:

Componente 1: Laboratorio1 Recepcao CorredorA Laboratorio2 Auditorio

Componente 2: CorredorB Escada Saida

Total de componentes conexas: 2.

5. Observações e conclusões

Como compilar e executar

A partir do diretório lab7:

```
gcc -Wall -Wextra grafo.c main.c -o lab7
./lab7
```

O programa não lê arquivos de entrada: o mapa do edifício é construído diretamente no código (`constroi_mapa`), e todos os experimentos já estão fixados em `main`.

Facilidades e dificuldades

- O uso de **vértices nomeados** (em vez de só índices) deixou a saída muito mais legível para o contexto de evacuação, ao custo de uma pequena tradução nome→índice (`indice_vertice`).
- Foi preciso atenção ao **grafo não dirigido**: toda aresta é inserida (e removida) nas duas listas de adjacência. Esquecer um dos lados quebraria a simetria e os algoritmos de busca.
- Distinguir **DFS × BFS** foi o ponto conceitual central: a DFS é simples e recursiva, mas só a BFS (com fila) garante o caminho mínimo em grafos não ponderados.
- A contagem de **componentes conexas** reaproveita a ideia das buscas: enquanto houver vértice não visitado, inicia-se uma nova busca, e cada busca corresponde a uma componente.

Resultados dos testes

O programa **compila sem warnings** (`-Wall -Wextra`) e **funciona** em todos os blocos pedidos: monta corretamente as listas de adjacência, a DFS visita os 8 ambientes no grafo original, a BFS devolve os caminhos mínimos de 4 e 5 deslocamentos, e após o bloqueio o grafo se divide em 2 componentes conexas, deixando Corredor B, Escada e Saída isolados da Recepção. Não foram observados problemas; toda a memória alocada é liberada ao final com `libera_grafo`.